

Đặc tả: Hệ thống Thông báo (Notification & Message Queue)

1. Mô tả

Hệ thống Thông báo chịu trách nhiệm gửi các thông tin quan trọng (như xác nhận đăng ký thành công, vé QR, thông báo hủy sự kiện) đến sinh viên. Tính năng này được thiết kế tách biệt hoàn toàn khỏi luồng API chính bằng kiến trúc **Message Queue (Hàng đợi thông điệp)**. Đồng thời, code được cấu trúc theo **Strategy Pattern (Mẫu thiết kế Chiến lược)**, đảm bảo hiện tại có thể gửi Email (qua Mailtrap), nhưng trong tương lai có thể dễ dàng thêm các kênh mới như Telegram, Zalo hay SMS mà không làm ảnh hưởng đến mã nguồn cũ.

2. Luồng chính (Happy Path)

Bối cảnh: Một sinh viên vừa đăng ký thành công workshop (hoặc thanh toán thành công). Hệ thống cần gửi vé QR cho sinh viên.

B1. Đẩy thông điệp (Produce): Core Backend API (luồng đăng ký) khởi tạo một đối tượng Payload chứa: user_email, full_name, workshop_info, và qr_code_hash. API này đẩy Payload vào một hàng đợi trên Redis có tên là notification_queue thông qua thư viện BullMQ, sau đó lập tức trả về 200 OK cho người dùng mà không cần chờ email gửi xong.

B2. Nhận thông điệp (Consume): Background Worker (chạy ở một process riêng) liên tục lắng nghe notification_queue. Nó lấy Job vừa được đẩy vào để bắt đầu xử lý.

B3. Phân luồng kênh gửi (Routing): Dịch vụ Thông báo (Notification Service) đọc cấu hình hệ thống. Nhận thấy kênh gửi hiện tại là EMAIL, nó khởi tạo lớp EmailProvider (kế thừa từ giao diện chung INotificationProvider).

B4. Biên dịch Template (Templating): Lớp EmailProvider nhúng các biến (full_name, qr_code_hash) vào một file template HTML thiết kế sẵn cho đẹp mắt.

B5. Gửi thông báo (Delivery): Sử dụng Nodemailer kết nối với SMTP của Mock Notification (Mailtrap) để thực hiện gửi email.

B6. Ghi nhận thành công (Acknowledge): Worker nhận được phản hồi 250 OK từ Mailtrap. Nó đánh dấu Job là COMPLETED trong BullMQ và ghi log hệ thống.

3. Kịch bản lỗi (Edge Cases & Exception Handling)

3.1. Máy chủ Email bị sập hoặc Timeout (Provider Down)

- **Trigger:** Dịch vụ SMTP của Mailtrap (hoặc Gmail/SendGrid sau này) gặp sự cố, không thể kết nối.
- **Xử lý:** BullMQ tự động bắt (catch) lỗi kết nối. Nó sẽ không đánh dấu Job là thất bại ngay, mà chuyển sang cơ chế **Exponential Backoff** (Thử lại sau 1 phút, 3 phút, 5 phút). Nếu sau 5 lần thử vẫn thất bại, Job sẽ được chuyển vào Dead Letter Queue - DLQ để Admin vào xem xét thủ công. Không có dữ liệu nào bị mất.

3.2. Sai định dạng Email (Invalid Address)

- **Trigger:** Mặc dù tài khoản được đồng bộ tự động từ hệ thống cũ, nhưng do dữ liệu gốc bị “bẩn” (Ví dụ trong file CSV của trường, nhân vụ đào tạo gõ nhầm email sinh viên thành nguyenvana@gmailcom - thiếu dấu chấm). Dịch vụ SMTP từ chối gửi.
- **Xử lý:** Lỗi này là lỗi từ phía dữ liệu cứng, việc thử lại (Retry) cũng vô ích. Worker phân tích mã lỗi trả về từ Mailtrap (VD: 5xx Syntax error), lập tức đánh dấu Job là FAILED (Bỏ qua bước Exponential Backoff) và ghi log cảnh báo để Admin biết và cập nhật lại thông tin sinh viên. Không làm nghẽn các email hợp lệ khác trong hàng đợi.

3.3. Tải đột biến hàng vạ thông báo (Queue Spike)

- **Trigger:** 12.000 sinh viên đăng ký thành công trong vòng 10 phút, tương đương 12.000 Jobs bị đẩy vào Queue cùng lúc.
- **Xử lý:** Core API chỉ mất 1-2ms để nhét 1 Job vào Redis, nên giao diện người dùng không bị treo. Redis đủ sức chứa hàng triệu Job trên RAM. Background Worker sẽ từ từ lấy từng Job ra gửi (ví dụ giới hạn tốc độ: gửi 50 emails/giây để tránh bị nhà cung cấp email đánh dấu là Spammer). Sinh viên có thể nhận email chậm hơn 1-2 phút, nhưng hệ thống tuyệt đối an toàn.

4. Ràng buộc (Constraints)

- **Tính rời rạc (Decoupling):** Luồng chính của API (Đăng ký/Thanh toán) tuyệt đối không được dùng lệnh `await sendEmail()`. Bắt buộc phải dùng `await queue.add()`.
- **Nguyên lý Đóng/Mở (Open/Closed Principle):** Để thiết kế sẵn cho tương lai, mã nguồn phải định nghĩa một Interface `NotificationChannel` có hàm `send(payload)`. Mọi nhà cung cấp (Email, Telegram) đều phải implements Interface này. Sau này thêm Telegram, chỉ việc code thêm file `TelegramProvider.js` mà không phải sửa bất kỳ dòng code cốt lõi nào của API Đăng ký.
- **Môi trường Demo:** Toàn bộ email trong môi trường phát triển (Dev/Local) phải được chặn và chuyển hướng vào Mailtrap để tránh việc vô tình gửi hàng ngàn email rác ra ngoài thế giới thực, dẫn đến việc IP bị đưa vào Blacklist.

5. Tiêu chí chấp nhận (Acceptance Criteria)

- **Test Case 1 (Decoupling Performance):** Đẩy 1.000 Job thông báo vào Queue. Thời gian phản hồi trung bình của API đẩy Job phải duy trì dưới 10ms.
- **Test Case 2 (Email Delivery):** Đăng ký thành công một workshop. Truy cập vào dashboard của Mailtrap, thấy một email xuất hiện với giao diện HTML chuẩn, chứa đúng họ tên sinh viên và link hình ảnh QR Code.
- **Test Case 3 (Retry Mechanism):** Tạm thời đổi mật khẩu/port SMTP sai để giả lập lỗi Provider. Thực hiện đăng ký. Quan sát log của BullMQ thấy hệ thống tự động báo lỗi và lên lịch thử lại (Retry) chính xác theo cấp số nhân (ví dụ: sau 1p, 2p, 4p).